
Rethinking Benchmarking Trajectory Error Accumulation in Fluid Dynamics Datasets

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 A recent body of work has developed fully-learned fluid simulators. Methods
2 to train those include single-step training (teacher-forcing), or using unrolled
3 training. For single-step approaches, recent papers have highlighted the importance
4 of capturing high-frequency components to ensure high-fidelity autoregressive
5 rollouts over long horizons. The question we aim to answer is to what extent each
6 of those approaches relies to the long-term behaviour of the autoregressive rollout.
7 In other terms, we wish to find out what are the main drivers for the drift of a
8 trajectory, and how this differs for existing models. Overall, a formalization of
9 what is the drift and which component are important to reduce it is missing.

10 1 Introduction

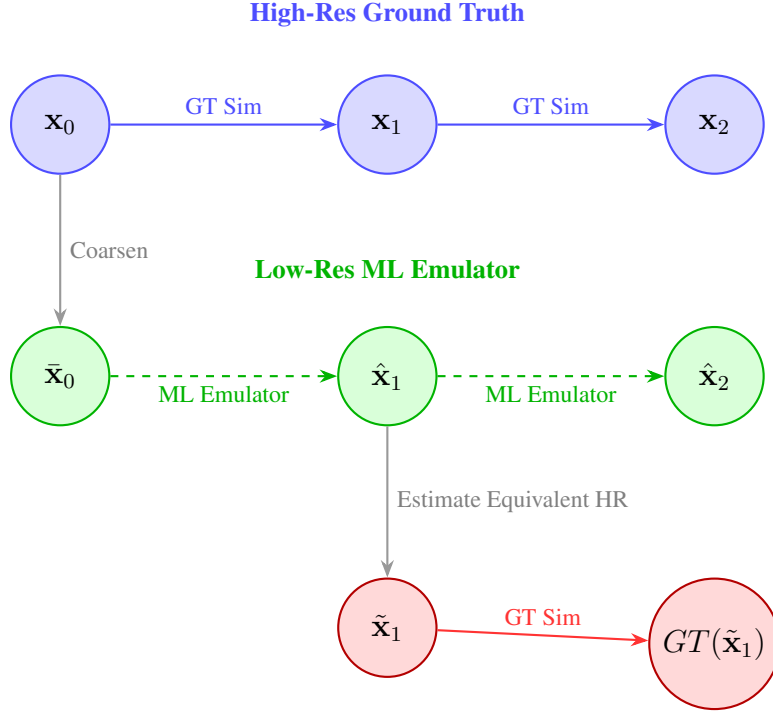
11 What leads an autoregressive-model to drift away from a ground-truth trajectory ? What is the role
12 of the single-step prediction in the accumulation of error ? How do such dynamics depend on the
13 dataset in question ? How do different models accumulate different types of errors ? Those are the
14 questions we aim at answering.

15 2 Motivation : Why is that important ?

- 16 • First of all, to better validate the claimed properties of a new model. For instance, if a
17 model is claimed to better capture the high-frequencies, one should be able to show that the
18 long-term behaviour actually benefits from better captured high-frequencies.
- 19 • To identify the remaining challenges / parts to improve on a given (dataset, model pair): If
20 the model suffers from exposure bias (going out-of-distribution and performance suffering
21 from this), there should be a specific solution for this. If there is no exposure bias, the
22 one-step prediction and how it emphasizes some frequencies w.r.t others should be the most
23 important part. If it's a conjunction of both, how to address those?
- 24 • Currently, most works look at: MSE over the rollout / Time of high correlation with the
25 ground-truth, and then validate that the physical consistency is maintained by looking at
26 metrics such as TKE, evolution of the energy spectrum. The latter however don't give a
27 diagnosis on which aspects the model is better than the latter: is it short-term accuracy, no
28 exposure-bias, better captured frequencies ?
- 29 • To know what to expect from a model on a new dataset, as well as for reproducibility: by
30 looking only at the MSE of the model, a new practitioner/researcher doesn't know how a
31 model should behave/easier debugging

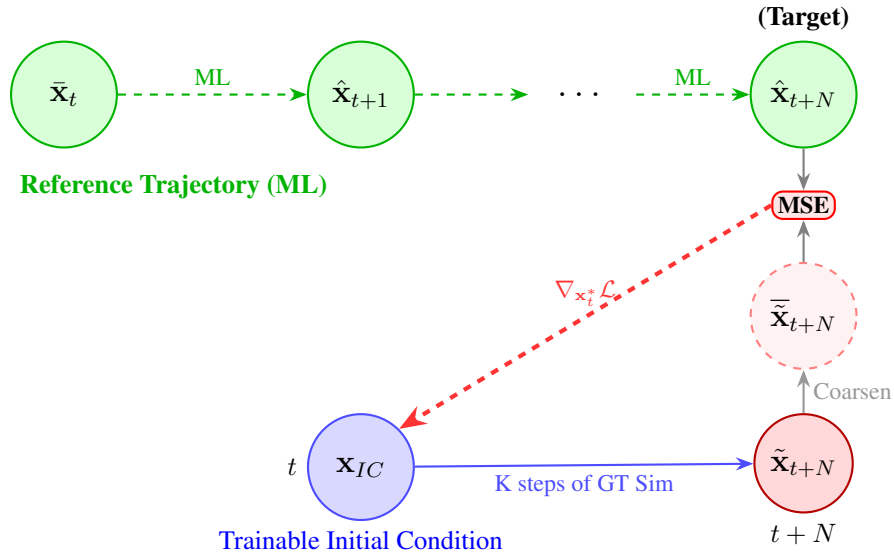
one-step error dependent, are not suitable for datasets where the main error driver is exposure bias and systematic drift, such as Transonic Flow).

4.3 Obtaining a proxy for the model Ground-truth prediction



71

The estimate $\tilde{x}_1$ must be very precise, in order for the GT transition to be accurate. A super-resolution operation based only on $\hat{x}_1$ is intuitively not sufficient. Therefore, our approach consists in optimizing the initial condition of a chain of high-resolution simulations to make the final coarsened state match the low-resolution model prediction.



76

Using more GT Sim calls leads to a more physically-plausible state (it acts as a regularizer, i.e. after only a few simulator calls, the output may still contain non-physical artifacts), but it's also more expensive, and potentially harder to optimize. Also since the problem is under-determined (the target

is a low-resolution field for which there may be infinitely-many relevant high-resolution fields). There regularization terms could also be explored to constrain the space of high-resolution states.

The main issue of this method is that it is slow: it runs backpropagation operations at high-resolution. It is furthermore required to run this optimization for every datapoint, and every autoregressive call. It also requires a high amount of GPU RAM (already 30 GB for batch-size of 5 and 2 simulator calls)... If one only uses this approach post-training as an analysis tool, this is probably fine.

As an alternative to retraining an initial condition for every (data index, AR step) pair, one could train a neural network to map the initial condition to the refined one (this would be less memory expansive, at the expense of being accurate). Note that in this problem we actually want to overfit, since we want to exactly match the model's predicted state.

5 The trajectory drift

Assuming that we can write the

The trajectory drift is influenced by the "shape" of the error. In other words, starting from a same initial condition X_t , two trajectories from two different neural operators $\hat{X}_{t+1}^{(1)}$ and $\hat{X}_{t+1}^{(2)}$ with a same one-step error $(d_{OS}^{(1)}(t) = d_{OS}^{(2)}(t))$, may have a different next-step trajectory drift : $d_{TD}^{(1)}(t+1) \neq d_{TD}^{(2)}(t+1)$. This is because a given trajectory may diverge faster from the ground-truth trajectory depending on which errors are made compared to that trajectory.

5.1 The Pareto-front of frequency errors : Highlighting the trajectory drift effect

Assuming that a neural emulator has a fixed one-step error, and no exposure bias. We aim at identifying the response of the trajectory drift to the shape of the one-step error. Since we once again don't have access to a ground-truth simulator to which one could inject an error of a given shape, we make use of two neural emulators, and look at the linear combination between them.

We look at applying transformations to the two model's predictions

6 Finding the optimal trade-off between Exposure-Bias and Performance

As mentioned, it is not sufficient to look at training process error, as it could lead to exposure bias. Furthermore, it is also not sufficient to look at the sampling process error, as the reduction of error in a given timestep may come from reduction in other timesteps. Our aim is to isolate the effect that a given timestep will incur in the next timesteps, as exposure bias. in previous timesteps directly impact error in current timestep.

We highlight a few points:

- if a model has exposure bias then it is likely that
- How close the input error needs to be
- How much of the "exposure-bias" comes from the fact that the model is not supposed to be deterministic ? Rather than drifting ?

6.0.1 Model Cross-Comparisons : A proxy metric for exposure bias

The intuition is the following: Given the trajectory $\{\hat{x}_1, \dots, \hat{x}_K\}$ obtained from autoregressive sampling of NO_1 , if both models have a low exposure bias, then the average distance between $\hat{x}_t$ and $NO_2(\hat{x}_{t-1})$ should not vary much as a function of t . Note that this is only a sufficient condition: if the average distance between NO_1 and NO_2 stays constant, it doesn't imply that they are both exposure-bias free.

By using multiple "reference" (evaluation) models, we can (potentially?) get rid of the case where the distance between models doesn't increase with constant distance. We may also be able to obtain a more accurate estimate of each model's biases.

123 In particular, we look at the evolution of:

$$\mathbb{E}\|\hat{x}_t - NO_2(\hat{x}_{t-1})\|^2 = \mathbb{E}[\epsilon(x_t)^2 + \|NO_2(\hat{x}_{t-1}) - GT(\hat{x}_{t-1})\|^2] \quad (5)$$

$$- 2\langle \hat{x}_t - GT(\hat{x}_{t-1}), NO_2(\hat{x}_{t-1}) - GT(\hat{x}_{t-1}) \rangle \quad (6)$$

$$= \mathbb{E}\left[(\epsilon_{NN_1}(x_0) + EB_{NN_1}(\hat{x}_{t-1}))^2\right] \quad (7)$$

124 The second term is the single-step error made by the second neural-operator. It can also be de-
125 composed as a sum of the systematic drift of NO_2 , and the exposure-bias of NO_2 when using the
126 trajectory of NO_1 as input.

127 **In practice**, we observe that a cross-correlation doesn't necessarily induce a faster increase of error.
128 The reason could be two things:

- 129 • The cross-correlation is not a good indicator of the exposure bias ? Indeed, the cross-
130 correlation implies that the samples generated by the neural emulator become out-of-
131 distribution. It however doesn't necessarily imply that the model suffers from this drift. Still,
132 is it possible that two models get further from each other and their performance does not
133 degrade ? Can we think about 2-steps metric to show that a model indeed suffers from its
134 own drift ? Like Eval(M1(M1(x))) vs M1(M1(x)) (regarde carnet).
- 135 • If use one model to evaluate, could just mean that it is more sensible to a certain type of
136 drift than another. Can either : evaluate with more models. Actually, the cross-comparison
137 of a diffusion model with a certain schedule and a diffusion model with another schedule is
138 non-increasing (it even decreases).
- 139 • The cross-correlation is a good indicator of the exposure bias, but exposure bias becomes
140 insignificant in the error accumulation (due for instance, to dynamics). In particular, at the
141 time where exposure bias is too high, the trajectory has maybe drifted so much that the error
142 cannot be seen.
- 143 • How to explain that evaluating a diffusion model traj. with a diffusion model evaluator leads
144 to a reduction of error as a function of the simulation step ?
- 145 • The evaluator is the one that suffers from the out-of-distribution : the discrepancy at a given
146 timestep could hinder the evaluator rather than the model itself.

147 6.1 Ideas of cross-comparisons evaluations

- 148 • On the trajectory of M_1 , apply M_2 (with less exposure bias, for instance a 2-steps Unet
149 which has no out-of-distribution samples). We assume that this will "put back" the sample
150 more in-distribution. Then apply M_1 again, and compare to applying M_1 twice.

151 6.2 The exposure bias of different neural emulators

152 7 Experiment log

153 7.1 Phases of the project

154 **PHASE 1: Identify how HR proxy emulation works** - Are there any troubles with it - What are
155 the metrics of interest to measure how good the estimate is - What is the correct way to initialize the
156 initial condition (before optimization) : - initial condition may influence how easy it is to obtain a
157 correct or incorrect solution to the optimization problem - Explore how the dataset influences the
158 correctness of the estimate, w.r.t the defined metrics - Effect of the hyperparameters (number of proxy
159 simulator calls, number of optimization steps, type of input at initialization, regularization of the
160 prediction)

161 **There, the main point is ensuring the unconditional (dataset and model independent) correctness**
162 **of the proxy estimator. If the proxy is incorrect, the downstream uses of it are useless**

163 **PHASE 2: Exploring the potential downstream applications of the HR proxy emulation**

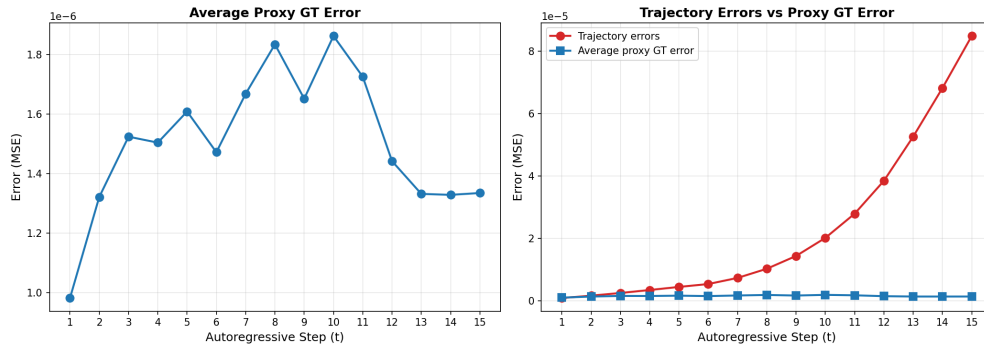
164 7.2 Logs

165 **13.02.2026** An important issue seems to be the computational overhead required to optimize the
 166 initial condition in high-resolution. For a batch size of 4, and 15 AR steps, 1000 steps of optimization
 167 take more than 10 hours. The question is whether we can juggle with the parameters (increasing
 168 model calls and decreasing number of steps) to cope with that. The issue is that we will anyway not
 169 be able to run the experiment on a batch of 64, let alone the full validation set.

170 → Attempt to cope with that : look at effect of number of optimization steps on accuracy at ts.1.
 171 Explore the NB STEPS x MODEL CALLS GRID

172 Result : for Kolmo, 5 calls seems to be a good starting point. Also, lower learning rate (0.001) leads
 173 to less explosion.

14.02.2026 Obtaining results on a single trajectory, for Diffusion Adaptive UT :



174

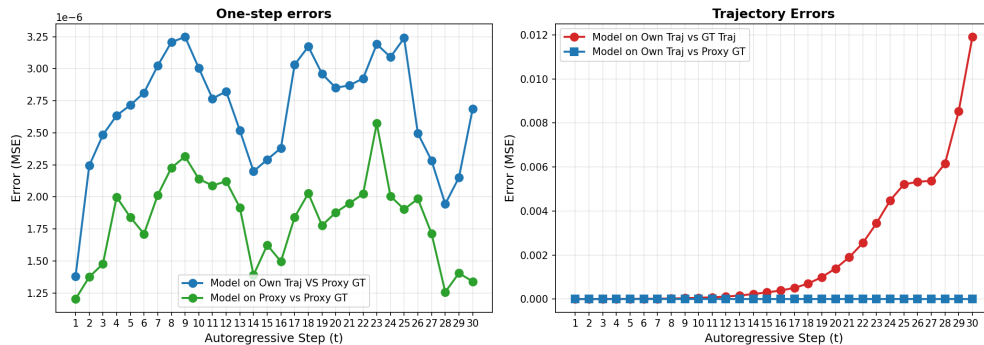
175 It's not sure whether the increase is due to :

- 176 • (1) Harder optimization at later steps
- 177 • (2) Harder One-step error depending on position on trajectory
- 178 • (3) Actual Exposure-Bias

179 (1) The type of initialization "model-traj" is good for steps larger than n simulator calls, but for earlier
 180 steps it still uses the GT traj, which could lead to unequal treatment. (2) Adding the Proxy GT vs
 181 Model on Proxy GT Error as a proxy should get rid of point. This should give a closer baseline, if the
 182 one-step error fluctuates based on the position in the trajectory.

183 Overall, a guarantee of the correctness of the Proxy is very important.

15.02.2026 Now with the Proxy GT vs Model on Proxy GT:



184

185 The two curves follow each other closely, showing the fluctuations are due to the fluctuations in the
186 trajectory rather than in exposure-bias / estimate correctness. This at least shows that the proxy is
187 able to consistently (in the same way) match the model trajectory (for timesteps ≥ 2).

188 We see an increase in the ratio between the two curves at timestep 2, and then close to no increase (at
189 eye view).

190 The question is whether this increase is due to exposure-bias, or due to bad calibration/convergence
191 of the proxy. Also, could just be that the distance is due to the fact that there is no high-resolution
192 proxy which can match the low-resolution model prediction.

193 **Might not be able to give a certificate, but at least can we put the initial prediction in "disad-**
194 **vantage", i.e. make initial condition further away from trajectory**

195 **16.02.2026** Meeting with Felix. Advices: start with 1D cases where we don't actually need to run
196 a coarsening operation. Simple cases would also allow to have a better characterization of what it
197 means to be "in-distribution" (Fourier approach). Furthermore, one can iterate faster (his advice is :
198 simulation should take in order of seconds, training should take in order of 10 minutes).

199 This makes sure that we have full vision (for instance, in the HR case, we only compare the LR fields,
200 so the HR potentially could be out-of-dist). Multi-resolution could be a next step.

201 So start with advection, KS even, Burgers. Or chaotic case where we can validate that drifting obeys
202 to chaos theory.

203 The plan is therefore to start with exponax, compute the initial condition for a lot of trajectories, get
204 the exposure-bias. Maybe we can also get a characterization of the error separation.

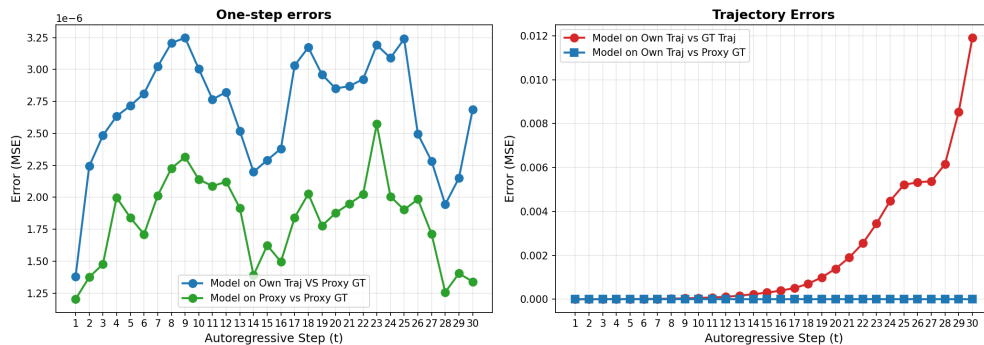
205 **18.02.2026** First runs on APEBench, 1D Advection, checking the different training options (one,
206 sup unrolling, diverted chain unrolling), with advection gamma equal to 10.5

207 On this case, multiple questions: - one has a better first-step mse than sup;2 and sup;5, which is
208 intuitive - however, one has a worse first-step mse than div;2 and div;5, which is not intuitive (check
209 myself what div means, and ask Felix if doesn't make sense still) - we also see that among one, sup;2
210 and sup;5, sup;5 has worse first-step MSE, but best long-term MSE, which is intuitive. Question is
211 whether this difference is due to exposure-bias reduction, or to predictive placement.

212 I think that, without diving into Exposure-Bias etc, the first hypothesis is that first-step MSE does
213 a big part of the job. This sounds obvious, but this is probably a finding to display. In the sense
214 that now, when creating a new architecture/model, one should display which part of the trajectory
215 improves. Check which proportion of the PDEs is defined by the one-step MSE,

216 There is also the hypothesis made in APEBench that unrolling could increasing the receptive field
217 and therefore lead to better one-step prediction ? Not sure though. It's also claimed that the diverted
218 training is a good trade-off between one-step MSE and long-term performance. Indeed, that's the
219 case. Using that method is therefore a matter of what's important for a given design in mind.

220 Ran experiments on FNO:



221 My hypothesis is that the only thing that architectures modify is the one-step MSE, in most cases.
222 Especially, when training them in one-step, architectures in general have the same error growth.

223 Actually, that’s not true for all architectures (and a regularization method can be thought of as being
 224 part of the architecture, for instance adding noise on input in diffusion models as in ACDM), but in
 225 most cases it is true. The question is rather, can we categorize architectures into categories of brought
 226 improvements. for instance that training with noiThey don’t bring an

227 Idea for the diffusion project: What about using different models, maybe also during inference. Could
 228 also vary the architectures, if it turns out that an architecture is better for a given noise range. This
 229 would increase the model size by 3 or 4, but in a smart way. And if we can show it improves the
 230 performance better than a classical ensemble, would be nice.

231 But for now, let’s say we want a single model: we can train a model fully on the lowest noise level,
 232 actually just pick the one with the lowest sampling error (rather than taking a parameter tau). Then
 233 we train earlier step, using binary search. And iterate. Or we train it all at the same time, and

234 8 Discussions with Claude

235 8.1 Frequency distribution of one-step error and its effect on rollout

236 A natural question is whether two models with the same total one-step MSE but different spectral
 237 error profiles will exhibit different rollout behaviour. By Parseval’s theorem, the MSE decomposes
 238 exactly into per-mode contributions:

$$\|\hat{x} - x\|^2 = \sum_k |\hat{x}_k - \tilde{x}_k|^2 \quad (8)$$

239 so at the single-step level, MSE is frequency-agnostic: an error of magnitude ϵ at wavenumber k_1
 240 contributes identically to an error of the same magnitude at wavenumber k_2 .

241 However, the picture changes drastically when we consider autoregressive rollout. The key observa-
 242 tion is that the *trajectory drift* $d_{TD}(t)$ depends not only on the magnitude of the one-step error, but
 243 on its spectral distribution.

244 **The general case: forward energy cascade.** In most 3D turbulent systems, energy cascades from
 245 large scales to small scales (forward cascade). In such systems:

- 246 • **Low-frequency errors** corrupt the large-scale structure of the flow. Since the large scales
 247 govern advection and the overall trajectory of the system, these errors propagate to all scales
 248 through nonlinear mode coupling. A wrong large-scale flow transports features to wrong
 249 locations, leading to rapid trajectory divergence.
- 250 • **High-frequency errors** are noise-like and tend to be damped by physical dissipation
 251 (viscosity). They remain spectrally localized for short times and have a weaker feedback
 252 onto the large-scale dynamics.

253 Therefore, a model whose one-step error is concentrated in low frequencies will typically exhibit
 254 faster trajectory drift than one whose error is concentrated in high frequencies, even at equal total
 255 MSE. This favours architectures like UNets, whose spatial locality and pooling hierarchy naturally
 256 capture low-frequency structure well, even if they smear high-frequency detail.

257 **The inverse cascade case: Kolmogorov flow.** Kolmogorov flow is 2D Navier-Stokes with sinu-
 258 soidal forcing at a fixed wavenumber, and is one of the most common benchmarks in the neural
 259 operator literature. Crucially, 2D turbulence exhibits an *inverse energy cascade*: energy injected at
 260 small scales flows upward to larger scales, while enstrophy cascades downward.

261 In this regime, the argument reverses. High-frequency errors can feed energy upward into the large-
 262 scale structure through the inverse cascade, eventually corrupting it. Getting the small scales right
 263 therefore matters for the large-scale trajectory, which means the FNO’s purported advantage at high
 264 frequencies could genuinely translate into better rollout stability on this specific benchmark.

265 Other systems exhibiting inverse cascades include rotating and stratified (geophysical) flows, thin
 266 fluid films, and certain MHD configurations.

267 **Role of the Reynolds number.** The Reynolds number $Re = UL/\nu$ controls the ratio of inertial to
 268 viscous forces and determines the width of the inertial range over which the cascade operates.

- 269 • At **low Re** (typical of many benchmarks, $Re \sim 100\text{--}1000$), viscosity damps fluctuations
 270 efficiently across scales. The inverse cascade has limited room to operate, and errors at small
 271 scales are dissipated before they can migrate significantly upward. The spectral distribution
 272 of error has a moderate effect on rollout.
- 273 • At **high Re** ($Re \gtrsim 10000$), the inertial range is wide and the inverse cascade is vigorous.
 274 High-frequency errors have many scales to climb through and can seriously corrupt large-
 275 scale structures during rollout.

276 This means that model comparisons are Reynolds-number dependent: a UNet and an FNO may
 277 show similar rollout behaviour at moderate Re, while at high Re the difference in their spectral
 278 error profiles becomes decisive. This should be taken into account when designing benchmarks and
 279 interpreting results: the Re at which a benchmark is run determines how much the spectral structure
 280 of the one-step error matters for long-term trajectory fidelity.

281 **Implications for benchmarking.** This analysis suggests that reporting only aggregate one-step
 282 MSE (or even rollout MSE) is insufficient. One should additionally report:

- 283 • The spectral decomposition of the one-step error (per-wavenumber MSE)
- 284 • The Reynolds number and cascade regime of the benchmark system
- 285 • Whether rollout improvements stem from better low-frequency or high-frequency accuracy

286 This would allow distinguishing whether a claimed architectural improvement (e.g., “better high-
 287 frequency capture”) actually benefits the downstream rollout task, or whether it merely redistributes
 288 error across frequencies without affecting trajectory drift.

289 **A 1D test case: Kuramoto-Sivashinsky.** The frequency-dependent rollout effect can also be
 290 observed in 1D, using the Kuramoto-Sivashinsky (KS) equation. The KS equation features a linear
 291 instability band at intermediate wavenumbers that injects energy, while the quadratic nonlinearity
 292 $u \cdot u_x$ redistributes energy across scales. Crucially, this redistribution goes *both directions*: forward to
 293 high wavenumbers (where the hyperdiffusion u_{xxxx} dissipates it) and backward to low wavenumbers.
 294 This partial inverse transfer means that high-frequency errors can feed into low- k modes through
 295 nonlinear interactions.

296 The linear growth rate per wavenumber k is:

$$\sigma(k) = \underbrace{|\gamma_2| k^2}_{\text{anti-diffusion (energy injection)}} - \underbrace{|\gamma_4| k^4}_{\text{hyper-diffusion (dissipation)}} \quad (9)$$

297 Modes are linearly unstable where $\sigma(k) > 0$, i.e. for $k < k_c$ where:

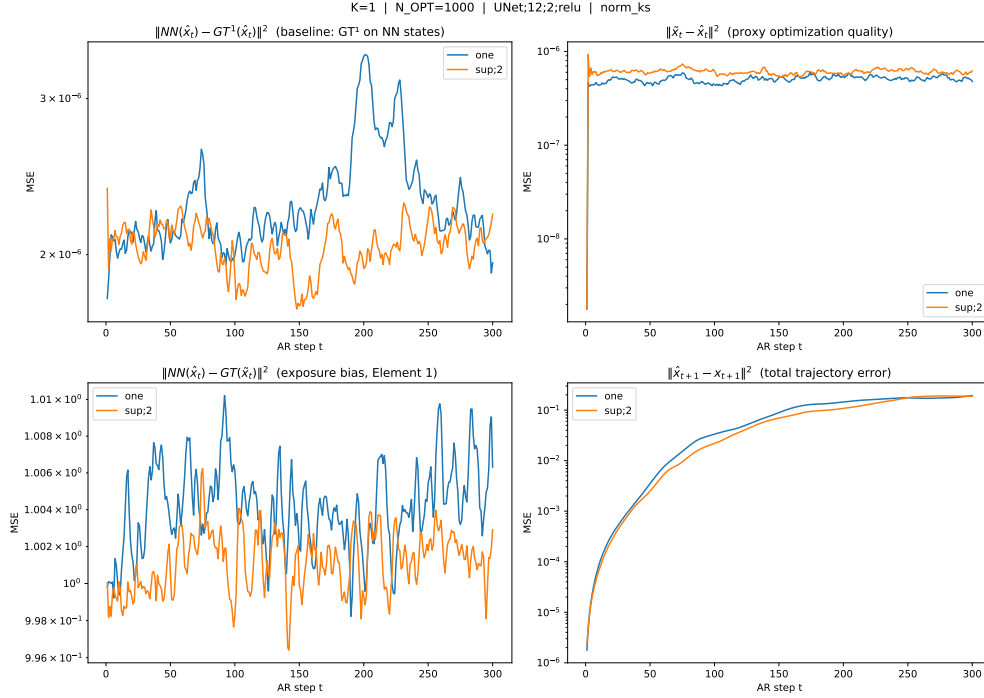
$$k_c = \sqrt{\frac{|\gamma_2|}{|\gamma_4|}} \quad (10)$$

298 With default parameters ($\gamma_2 = -1.2$, $\gamma_4 = -15.0$), this gives $k_c \approx 0.28$. To widen the instability
 299 band so that the task relies more on high-frequency accuracy, one increases the ratio $|\gamma_2|/|\gamma_4|$, pushing
 300 k_c to higher wavenumbers.

301 Additionally, the KS equation is chaotic (positive Lyapunov exponents), so any perturbation grows
 302 exponentially. However, the rate of growth depends on which modes are perturbed: errors in the
 303 linearly unstable band (the most dynamically active wavenumbers) are amplified fastest.

304 This makes KS a suitable 1D testbed for the spectral error experiment: given two models with the
 305 same total one-step MSE, one with error concentrated at low- k and one at high- k , we expect different
 306 rollout divergence rates. The chaotic dynamics should amplify the difference, and the partial inverse
 307 transfer should make high-frequency errors more consequential than in a purely forward-cascade
 308 system.

309 By contrast, Burgers’ equation is less informative for this analysis: it is not chaotic (shocks are
 310 attracting structures), so errors are either absorbed into shocks or advected away rather than cascading
 311 across scales.



10.03.2026 The issue with the approach is that the proxy GT is not able to match well the ML simulator trajectory, and the error is much lower for $t = 0$ than for even $t = 1$. For $t > 1$, it appears the error between the proxy GT and the ML simulation is stable for unrolled steps. See top right plot, which shows error between ML trajectory, and proxy step, for $K = 1$. Behaviour is similar for larger K .

Actually, let's try to not compare the model's prediction with the GT on the GT proxy, as it may be arbitrary far away. What we want to know is whether a ML model suffers from its own distribution more than it would suffer from a GT state. By suffering, we therefore mean that its error is further away from the GT trajectory (total trajectory error). Therefore, at each step in the ML trajectory, we can run the ML model on the proxy GT, the ML model on itself, and compute the 2 errors w.r.t the GT trajectory. Also one must compute the difference in error between $MSE(NN(x_{hat}), x_{gt})$ and $MSE(NN(x_{tilde}), x_{gt})$ to ensure that the cost of out-of-dist input is not just due to the proxy gt being closer to the true trajectory